EXPERT-ANCHORED REPLAY REFINEMENT FOR TASK-VECTOR MERGING

Anonymous authors
Paper under double-blind review

ABSTRACT

We study model merging when only a small labeled replay buffer is available for each task. We introduce expert-anchored gradient descent (XGD), which can start from any initial merge or directly from the pretrained model. For each task, a gradient step scaled by the distance to its expert is confined, coordinate-wise, to the interval between the current value and the expert's value: steps away from the expert are dropped, and steps past it are clipped. We evaluate the method on multi-head GLUE-8 and the eight- and twenty-task CLIP suites, with every method given the same budget of at most 32 labeled examples. From the pretrained model, the anchored update beats unconstrained descent on every benchmark and every buffer draw, and beats every checkpoint-only merge on the CLIP suites; from merge initializations, it typically improves performance. We further observe that it is good at retaining the pretrained model's ability.

1 INTRODUCTION

Merging fine-tuned variants of a shared pretrained model combines their skills in weight space, without extensive retraining, ensembling, or routing. But merges degrade under task interference. In many practical settings a small *labeled* replay buffer per task is available.

A naive way of using such small buffer is to simply do gradient descent, either on the pretrained model or an initial merge, but ordinary gradient descent can be risky when only a few examples are available.

We propose a refinement that exploits the position of the task experts, *expert-anchored gradient descent* (XGD): each coordinate takes a gradient step scaled by its remaining distance to the expert and confined to the interval between its current value and the expert's value—it moves only toward the expert, only where the replay gradient locally agrees, and never past it—from *any* initialization, including the pretrained model itself. The update is provably non-expansive toward the experts' box hull at any learning rate (Proposition 1).

Our contributions are: (i) a replay-refinement method—expert-anchored gradient descent, a distance-preconditioned descent step projected onto the interval between the current model and each task expert—with a divergence-safety guarantee; (ii) a uniform evaluation across three benchmark families (multi-head GLUE-8 and the eight- and twenty-task CLIP suites); (iii) a measurement of what the refinement costs the pretrained model on tasks never merged or refined: the anchored update retains more than every checkpoint-only merge.

2 RELATED WORK AND BACKGROUND

Merging from checkpoints. Model merging combines fine-tuned variants of a shared pretrained model into a single model in weight space. Model soups average compatible checkpoints (Wortsman et al., 2022), and task arithmetic represents fine-tuning as a *task vector* $\tau_t = \theta_t - \theta_0$ and merges as $\theta_0 + \sum_t \lambda_t \tau_t$ (Ilharco et al., 2022). Interference between task vectors—over-counted directions, sign and magnitude conflicts—is the central failure mode, and one family of repairs constructs the merge from the checkpoints alone, differing in the rule that suppresses it: TIES-Merging trims small-magnitude updates and merges only sign-consistent components (Yadav et al., 2023); DARE randomly drops most delta parameters and rescales the rest (Yu et al., 2023), generalized by DELLA (Deep et al.,

2024); Model Breadcrumbs removes negligible and outlier perturbations (Davari and Belilovsky, 2023); the magnitude variant of Localize-and-Stitch stitches sparse task-relevant regions back into the pretrained model (He et al., 2024); and, closest in optimization form to our update, DOGE models merging as adaptive projective gradient descent (APGD), learning a shared modification to the task vectors under a data-free Taylor objective with gradients projected orthogonally to an SVD-derived shared subspace (Wei et al., 2025). "From checkpoints" describes the construction, not the reported numbers: each method carries a few hyperparameters—a scaling coefficient, a trim density—that are conventionally chosen on held-out validation data (Ilharco et al., 2022; Yadav et al., 2023), a cost MergeBench finds to dominate that of merging itself (He et al., 2025); TIES is the exception that also reports a fixed default recipe for the no-validation case.

Merging with unlabeled data. A second family fits the merge on unlabeled inputs or model outputs: Fisher merging weights parameters by a Fisher information estimated from the models' outputs on a data sample (Matena and Raffel, 2022); RegMean solves each linear layer by regression on its input activations (Jin et al., 2023); AdaMerging learns merge coefficients by entropy minimization, in its standard form on the unlabeled evaluation inputs (Yang et al., 2024); and Task Arithmetic in Trust Region (TATR) uses task-loss gradients at the pretrained model to identify low-interference dimensions in which to merge (Sun et al., 2025). Twin-Merging, Task Vector Bases, FusionBench, and MergeBench extend and standardize this space (Lu et al., 2024; Zeng et al., 2025; Tang et al., 2024; He et al., 2025).

Repair with labeled data. When a few labels per task are available, prior work spends them on low-dimensional objects fixed at merge time: LM-Cocktail sets merging weights from each model's loss on a small labeled sample (Xiao et al., 2024); the learned variant of Localize-and-Stitch trains its sparse masks on labeled data (He et al., 2024); Activated Parameter Locating (APL) uses few-shot examples to estimate parameter-partition importance, which determines pruning ratios and model-level merge weights (Kong et al., 2024); task-arithmetic coefficients can be tuned per task on the labels instead of on a validation split (Ilharco et al., 2022); and Fisher weights can be estimated from labeled-loss gradients (Matena and Raffel, 2022). Two further uses come from outside the merging literature and serve here as baselines: refitting only the classifier head on the labeled sample, i.e. linear probing on the frozen merged encoder (Alain and Bengio, 2016; Kumar et al., 2022), and rehearsal—fine-tuning the whole model on a stored buffer of examples (Robins, 1995; Chaudhry et al., 2019)—which in our grids appears as ordinary replay gradient descent. Common to all of these, the labels either shape a handful of coefficients, masks, or head weights, or drive unconstrained descent with nothing tying the model to the experts. The present work sits between the two: it asks whether a small labeled replay buffer can repair an initial merge—or build multi-task ability directly from the pretrained model—while staying close to the known experts, evaluating labeled task gradients at the *evolving* model and taking sequential, coordinate-wise steps toward the experts sized by first-order estimates of their effect. Section 4 orders all of these baselines by how much label information can reach the reported weights.

First-order intervention scores. Scoring a proposed parameter change by a first-order Taylor estimate—the inner product of a loss gradient with the change—appears in mechanistic interpretability as attribution patching (Syed et al., 2024; Kramar et al., 2024) and, closest to our setting, in APL, which replaces fine-tuned parameter partitions with their pretrained values and approximates their importance by the magnitude of a task-vector–gradient inner product at the pretrained model (Kong et al., 2024). APL's static, partition-level scores determine pruning ratios and merge weights. Our method instead evaluates signed, coordinate-level directional derivatives at the evolving model and turns them into sequential, clipped steps from the current initialization toward each task expert. Extended positioning against the benchmark families of the merging literature is given in Appendix C.

3 METHOD

3.1 PROBLEM SETUP

Let $\mathcal{T} = \{1, \ldots, T\}$ be a set of tasks whose experts are all initialized from the same pretrained encoder $\theta_0 \in \mathbb{R}^d$; fine-tuning on task t gives expert parameters θ_t and task vector $\tau_t = \theta_t - \theta_0$. Writing $r \in \{1, \ldots, d\}$ for a parameter coordinate, the refinement starts from an initial model

$\theta^{(0)} \in \mathbb{R}^d$: any weight-space merge of the experts—task arithmetic

$$\theta^{(0)} = \theta_0 + \sum_{i=1}^{T} \lambda_i \tau_i \tag{1}$$

Let $\mathcal{D}_t^{\text{probe}}$ be a small labeled replay buffer used for refinement, $\mathcal{D}_t^{\text{eval}}$ the evaluation split used only for reporting, and $\mathcal{L}_i(\theta; \mathcal{D}_i^{\text{probe}})$ the replay-buffer loss of encoder parameters θ under task i's prediction head. In the multi-head settings task-specific heads are not merged and the merged encoder is evaluated with each task's own head; the shared-output settings merge everything. The method uses no base-relative saliency term and no pre-merge pruning mask: its only data-dependent operation is a short replay-buffer refinement of $\theta^{(0)}$.

3.2 Expert-Anchored Gradient Descent

The naive way of exploiting the small replay buffer given either an initial merge or only the pretrained model is using it for gradient descent, $\theta^{(s+1)} = \theta^{(s)} - \eta \sum_{i=1}^{T} \nabla_\theta \mathcal{L}_i(\theta^{(s)}; \mathcal{D}_i^{\text{probe}})$, which treats the merge like any other starting point and lets the replay gradients move every coordinate freely. With only a small replay buffer, however, noisy gradients make such unrestricted movement risky and may permit only small learning rates and few steps. We instead use the positions of the experts to turn gradient descent into a safer, more targeted update.

The experts anchor the update. For current model parameters ϑ, let $v_i(\vartheta) = \theta_i - \vartheta$ be the coordinate-wise direction to expert i and let $g_i(\vartheta)$ be task i's replay gradient under its prediction head. Expert i is a parameter setting known to attain low loss on task i, so for each coordinate r the interval between the current value ϑ_r and the expert value $\theta_{i,r}$ is the natural admissible region for a task-i update: within it, the coordinate interpolates between the current model and one that works for the task, whereas a step beyond the expert, or away from it, has no such certificate. Our method—*expert-anchored gradient descent* (XGD)—is ordinary descent made to respect this region. For each task i and coordinate r, we (i) *precondition* the negative-gradient step by the current expert distance $|v_{i,r}|$, so that the expert's offset sets each coordinate's unit of movement and coordinates already agreeing with the expert barely move; and (ii) *project* the preconditioned step onto the admissible interval. The projection has two coordinate-wise effects: a step pointing away from the expert ($g_{i,r} v_{i,r} \geq 0$) returns zero, so the coordinate does not move—we call this the *gate*—and a step pointing past the expert stops at the expert value—the *expert-distance cap*. No update ever leaves the interval between the current value and the expert value.

Let $\theta^{(s,0)} = \theta^{(s)}$ and let π_s be the task order for sweep s. For within-sweep index $k \in \{1, \ldots, T\}$, set $i = \pi_s(k)$ and compute

$$g_i^{(s,k)} = \nabla_\theta \mathcal{L}_i(\theta^{(s,k-1)}; \mathcal{D}_i^{\text{probe}}), \tag{2}$$

$$v_i^{(s,k)} = \theta_i - \theta^{(s,k-1)}. \tag{3}$$

The update moves each coordinate that passes the gate a capped fraction of its remaining distance to the expert, and leaves every other coordinate unchanged:

$$u_{i,r}^{(s,k)} = \begin{cases} \min\big(\eta\, |g_{i,r}^{(s,k)}|,\ 1\big)\, v_{i,r}^{(s,k)} & \text{if } g_{i,r}^{(s,k)}\, v_{i,r}^{(s,k)} < 0, \\ 0 & \text{otherwise;} \end{cases} \tag{4}$$

the model is updated immediately, $\theta^{(s,k)} = \theta^{(s,k-1)} + u_i^{(s,k)}$, with $\theta^{(s+1)} = \theta^{(s,T)}$ after all T tasks in the sweep have been processed. Equation (4) is the projected, distance-preconditioned step written in one line: on coordinates that pass the gate, $-g_{i,r}^{(s,k)}$ and $v_{i,r}^{(s,k)}$ agree in sign, so the preconditioned descent step $-\eta\, g_{i,r}^{(s,k)} |v_{i,r}^{(s,k)}|$ equals $\eta\, |g_{i,r}^{(s,k)}|\, v_{i,r}^{(s,k)}$, and the $\min(\cdot, 1)$ factor is the projection onto the admissible interval. Its magnitude can equivalently be written as

$$|u_{i,r}^{(s,k)}| = \min\big(\eta\, |a_{i,r}(\theta^{(s,k-1)})|,\ |v_{i,r}^{(s,k)}|\big), \qquad a_{i,r}(\vartheta) := g_{i,r}(\vartheta)\, v_{i,r}(\vartheta).$$

The product $a_{i,r}$ has a meaning of its own: it is coordinate r's term in the directional derivative $\langle g_i(\vartheta), v_i(\vartheta) \rangle$, i.e. the first-order estimate of the loss change from the single-coordinate patch that sets ϑ_r to the expert value $\theta_{i,r}$—it attributes the predicted effect of adopting the expert wholesale to the

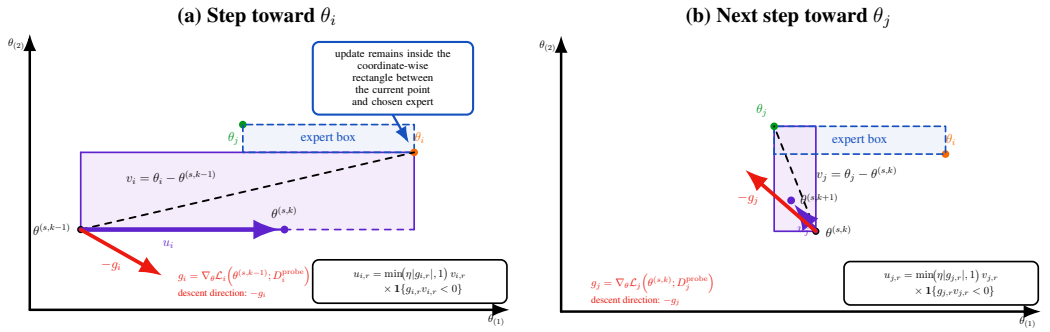

Figure 1: Sequential expert-anchored updates. (a) Starting from $\theta^{(s,k-1)}$, the task-i update u_i moves coordinate-wise toward θ_i and produces $\theta^{(s,k)}$. (b) The subsequent task-j update u_j moves from $\theta^{(s,k)}$ toward θ_j and produces $\theta^{(s,k+1)}$. Each violet rectangle is the coordinate-wise admissible region between the current iterate and the selected expert; the dashed blue rectangle is the expert box spanned by θ_i and θ_j.

individual coordinates. XGD thus moves exactly the coordinates whose patch is predicted to reduce the loss ($a_{i,r} < 0$; throughout, we call this sign condition the *attribution gate*), and never beyond the patch whose effect the estimate describes. The estimate certifies only a local first-order effect for task i; cross-task interference must be assessed empirically. Figure 1 illustrates two consecutive task updates.

Because the step fraction $\min(\eta\,|g_{i,r}^{(s,k)}|, 1)$ never exceeds one, per-coordinate movement is bounded by $|v_{i,r}^{(s,k)}|$ for any η: the learning rate controls how many coordinates reach the cap, never the maximum step. Optionally, $u_i^{(s,k)}$ is RMS-normalized tensor-wise. Algorithm 1 summarizes the procedure.

Algorithm 1 Sequential expert-anchored gradient descent (XGD)

Require: Task experts $\{\theta_i\}_{i=1}^T$; task heads; replay buffers $\{\mathcal{D}_i^{\text{probe}}\}_{i=1}^T$; initial model $\theta^{(0)}$ (any weight-space merge of the experts, or the pretrained model θ_0 itself); steps S; learning rate η.

for $s = 0, \ldots, S-1$ **do**
  $\theta^{(s,0)} \leftarrow \theta^{(s)}$.
  Choose a task order π_s over $\{1, \ldots, T\}$.
  for $k = 1, \ldots, T$ **do**
    $i \leftarrow \pi_s(k)$.
    $g_i^{(s,k)} \leftarrow \nabla_\theta \mathcal{L}_i(\theta^{(s,k-1)}; \mathcal{D}_i^{\text{probe}})$ using task i's head.
    $v_i^{(s,k)} \leftarrow \theta_i - \theta^{(s,k-1)}$.
    for each mergeable coordinate r **do**
      $u_{i,r}^{(s,k)} \leftarrow \min\big(\eta\,|g_{i,r}^{(s,k)}|,\, 1\big)\, v_{i,r}^{(s,k)} \cdot \mathbf{1}\{g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < 0\}$. {gated, capped step toward θ_i; Eq. (4)}
    end for
    $\theta^{(s,k)} \leftarrow \theta^{(s,k-1)} + u_i^{(s,k)}$.
  end for
  $\theta^{(s+1)} \leftarrow \theta^{(s,T)}$.
end for

return $\theta^{(S)}$.

The informal claim that the refinement "cannot diverge" is made precise by the following guarantee, whose proof (as an instance of a general geometric result, Theorem 1) is given in Appendix A.

Proposition 1 (Non-expansiveness toward the expert box)**.** *Let* $B_{\text{exp}} := \prod_{r=1}^d [\min_{j\in\mathcal{T}} \theta_{j,r},\ \max_{j\in\mathcal{T}} \theta_{j,r}]$ *be the axis-aligned box hull of the task experts, and let* $d_p(\cdot, B_{\text{exp}})$ *denote* ℓ^p *distance to it. Then every within-sweep update of Algorithm 1 is non-expansive*

in distance to the expert box: for every $1 \le p \le \infty$ and every sweep s and within-sweep index k,

$$d_p\Big(\theta^{(s,k)}, B_{\exp}\Big) \le d_p\Big(\theta^{(s,k-1)}, B_{\exp}\Big),$$

and hence $d_p(\theta^{(S)}, B_{\exp}) \le d_p(\theta^{(0)}, B_{\exp})$ after any number of sweeps, at any learning rate.

Every gated, clipped step moves each coordinate between its current value and the corresponding expert coordinate, and such movement can never increase the distance to the box; the box is moreover exactly the ℓ^1-geodesic convex hull of the experts (Proposition 2), not an arbitrary safe region. The guarantee holds for any initialization $\theta^{(0)}$, including the pretrained model.

4 Experimental Setup

Settings. We evaluate on three benchmark families. *Multi-head* GLUE-8 (CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, RTE) (Wang et al., 2018) with RoBERTa-base (Liu et al., 2019) merges the shared encoder and evaluates with each task's own trained head—an intentionally *oracle-task-id* setting. A *CLIP/ViT-B/32* vision track uses the standard eight-task task-vector suite (Radford et al., 2021; Ilharco et al., 2022) and its twenty-task extension (Wang et al., 2024), each task keeping its frozen text-derived zero-shot head; it probes a different modality and—since more task vectors then compete for the same encoder weights—a substantially higher-interference regime (Section 5.1). Heads are never fine-tuned on either track.

Protocol. Every method receives the same budget of B labeled examples per task ($B \in \{16, 32\}$), drawn from the training data and disjoint from the evaluation split (Appendix C specifies the sampling rule), and never sees a label outside it. Merge baselines choose their hyperparameters (coefficient, density, scale) on all B examples. XGD and its ablations split the budget in half: they refine on one half for a fixed horizon of $S = 50$ sweeps, choose the learning rate η on the other half, and are then refit once on all B examples at the chosen η. Selection everywhere uses the tasks' own held-out metrics, averaged over tasks, with ties going to the smaller learning rate (for a merge, the smaller scaling coefficient); grids are extended until the selected cell is interior, and the evaluation split is read only for selected cells. Every cell is repeated over three independent buffer draws and reported as mean $\pm$ standard deviation; the draws are fresh, the selection rule itself having been settled on three earlier diagnostic draws (Appendix **??**). Appendix C gives the grids and the remaining details.

Baselines. We compare against checkpoint-only merges (task arithmetic, TIES, DARE-TIES, Breadcrumbs, DOGE/APGD), merges fitted on unlabeled inputs (AdaMerging, RegMean; given the same B inputs per task that XGD sees labeled), and two other ways of spending the same labels: Fisher merging with a Fisher estimated on the replay buffer, and Localize-and-Stitch with masks learned on it (Appendix C). XGD's two ablations—ordinary replay gradient descent (GD; no anchor, no gate) and the ungated distance-scaled update (anchor without gate)—share XGD's optimizer, schedule, and grid, and are run from the pretrained model, where they isolate what each ingredient contributes; from merge initializations only XGD is run.

Metrics. The headline aggregate is the plain mean over tasks of each task's own metric—top-1 accuracy on the CLIP suites; Matthews correlation (CoLA), the mean of Pearson and Spearman correlation (STS-B), the mean of accuracy and F1 (MRPC, QQP) and accuracy elsewhere on GLUE-8, the benchmark's own convention—with the zero-shot floor and the expert ceiling quoted for scale. Worst-task values accompany every mean, since a merge can raise the average while harming one task badly; on GLUE-8 the worst task is reported on its own metric's scale and is usually CoLA's Matthews correlation. Normalized retention, which rescales each task between the pretrained model and its expert (Appendix C), is reported per task in the appendix only: as an aggregate it lets a task whose expert barely improves on the pretrained model dominate the average, so every table here reports the plain mean.

5 Results

Table 1: **All three suites at B=32** (32 labels per task for every method; written $\mathrm{mean}_{\pm\mathrm{std}}$ over three fresh buffer draws). Each suite reports the plain mean of its tasks' own metrics and the worst task on the same scale (Section 4); expert ceilings are 0.903 (CLIP-8), 0.860 (GLUE-8) and 0.896 (CLIP-20). A "+XGD" row refines that draw's own selected merge and is directly comparable to the merge row above it. Every method, merges included, selects its hyperparameters inside the budget by the same held-out-metric rule, so a merge row carries a draw spread wherever its selection varied. GD and ungated are XGD's ablations, run from the pretrained model where they isolate the anchor and the gate. The last block spends the same B labels on estimating merge weights instead of on refinement. Zero-shot floors are the θ_0 row.

	CLIP-8		GLUE-8		CLIP-20	
Model	Mean	Worst	Mean	Worst	Mean	Worst
Pretrained θ_0	0.478	0.315	0.354	−0.047	0.550	0.100
Task arithmetic	$0.685_{\pm 0.011}$	$0.503_{\pm 0.026}$	$0.552_{\pm 0.000}$	$0.130_{\pm 0.000}$	$0.604_{\pm 0.000}$	$0.115_{\pm 0.000}$
+XGD	$0.765_{\pm 0.011}$	$0.557_{\pm 0.004}$	$0.681_{\pm 0.023}$	$0.059_{\pm 0.045}$	$0.664_{\pm 0.005}$	$0.208_{\pm 0.051}$
Breadcrumbs	$0.705_{\pm 0.000}$	$0.521_{\pm 0.000}$	$0.473_{\pm 0.015}$	$0.149_{\pm 0.012}$	$0.598_{\pm 0.009}$	$0.133_{\pm 0.024}$
+XGD	$0.766_{\pm 0.008}$	$0.556_{\pm 0.006}$	$0.686_{\pm 0.009}$	$0.096_{\pm 0.030}$	$0.674_{\pm 0.004}$	$0.260_{\pm 0.021}$
DARE-TIES	$0.708_{\pm 0.000}$	$0.486_{\pm 0.000}$	$0.563_{\pm 0.000}$	$0.268_{\pm 0.000}$	$0.520_{\pm 0.000}$	$0.202_{\pm 0.000}$
+XGD	$\mathbf{0.793}_{\pm 0.010}$	$0.556_{\pm 0.013}$	$\mathbf{0.699}_{\pm 0.006}$	$0.212_{\pm 0.028}$	$0.692_{\pm 0.004}$	$\mathbf{0.329}_{\pm 0.018}$
TIES	$0.732_{\pm 0.002}$	$0.524_{\pm 0.018}$	$0.549_{\pm 0.000}$	$0.234_{\pm 0.000}$	$0.619_{\pm 0.000}$	$0.194_{\pm 0.000}$
+XGD	$0.785_{\pm 0.007}$	$\mathbf{0.564}_{\pm 0.005}$	$0.690_{\pm 0.010}$	$\mathbf{0.363}_{\pm 0.028}$	$\mathbf{0.695}_{\pm 0.003}$	$0.319_{\pm 0.010}$
θ_0+GD	$0.546_{\pm 0.032}$	$0.368_{\pm 0.041}$	$0.521_{\pm 0.017}$	$0.111_{\pm 0.068}$	$0.573_{\pm 0.021}$	$0.103_{\pm 0.005}$
θ_0+ungated	$0.579_{\pm 0.009}$	$0.443_{\pm 0.012}$	$0.480_{\pm 0.060}$	$0.115_{\pm 0.074}$	$0.602_{\pm 0.005}$	$0.156_{\pm 0.011}$
θ_0+XGD	$0.751_{\pm 0.004}$	$0.524_{\pm 0.003}$	$0.612_{\pm 0.015}$	$0.133_{\pm 0.047}$	$0.663_{\pm 0.009}$	$0.267_{\pm 0.029}$
Fisher (replay)	$0.657_{\pm 0.017}$	$0.361_{\pm 0.012}$	$0.541_{\pm 0.032}$	$0.112_{\pm 0.170}$	$0.610_{\pm 0.002}$	$0.117_{\pm 0.007}$
Localize-and-Stitch	$0.662_{\pm 0.015}$	$0.487_{\pm 0.005}$	$0.487_{\pm 0.077}$	$-0.060_{\pm 0.221}$	$0.605_{\pm 0.003}$	$0.130_{\pm 0.015}$

5.1 Performance on the Merged Tasks

Task-level scores at the 8+8 budget are given in Appendix D; the full twenty-task campaign, with its diagnostics and budget grids, is in Appendix E.

The underlying task-level evaluation scores for the two eight-task encoder benchmarks at the 8+8 budget are reported in Appendix Tables 5 and 6.

Refining from the pretrained model. Tables 1 and 2 report all three suites under the protocol of Section 4: every method given the same B labels per task, every hyperparameter selected inside that budget by one rule, and every number a mean over three fresh buffer draws. With no merge to inherit, XGD is the strongest cold-start treatment on all three suites at $B = 32$—0.751 on CLIP-8 against 0.732 for the best checkpoint-only merge, 0.612 on GLUE-8 against 0.563, and 0.663 on CLIP-20 against 0.619—using B labels per task in place of the eight or twenty expert checkpoints a merge requires. Neither ablation is viable at this budget: ordinary descent and the ungated update reach 0.546/0.579 on CLIP-8, 0.521/0.480 on GLUE-8 and 0.573/0.602 on CLIP-20, so the anchored, capped step is worth 0.06–0.21 over unconstrained descent from the same initialization, and the anchor without the gate does not account for it. The worst-task columns qualify the GLUE-8 result: XGD's mean there is bought partly from its weakest task (0.133 against DARE-TIES's 0.268), and every selected cell sits in the high-displacement part of the grid.

At $B = 16$ the cold start is less stable. XGD still leads on GLUE-8 (0.565 against 0.554) and ties TIES on CLIP-20 (0.613 against 0.612), but on CLIP-8 it falls to 0.681 ± 0.050 against TIES's 0.734—the one cell of either table in which a checkpoint-only merge beats it. The spread is selection rather than the update: two draws select $\eta \in \{4, 8\}$ and score 0.63–0.67 while the third selects $\eta = 32$ and scores 0.73, and pinning $\eta = 32$ on the two draws that rejected it scores 0.725 and 0.732. Cells are constructed on $B/2 = 8$ examples per task, where the larger rate genuinely collapses on some draws, and refit on 16, where it is safe; selection is right about the models it can see and conservative about the model it will build (Appendix **??**). At $B = 32$ on the same suite XGD varies by ± 0.004 and selects the same interior cell on every draw. An earlier version of that experiment

Table 2: **All three suites at B=16** (16 labels per task for every method; mean ± std over three fresh buffer draws). Each suite reports the plain mean of its tasks' own metrics and the worst task on the same scale (Section 4); expert ceilings are 0.903 (CLIP-8), 0.860 (GLUE-8) and 0.896 (CLIP-20). A "+XGD" row refines that draw's own selected merge and is directly comparable to the merge row above it. Every method, merges included, selects its hyperparameters inside the budget by the same held-out-metric rule, so a merge row carries a draw spread wherever its selection varied. GD and ungated are XGD's ablations, run from the pretrained model where they isolate the anchor and the gate. The last block spends the same B labels on estimating merge weights instead of on refinement. Same conventions as Table 1.

	CLIP-8		GLUE-8		CLIP-20	
Model	Mean	Worst	Mean	Worst	Mean	Worst
Pretrained θ_0	0.478	0.315	0.354	−0.047	0.550	0.100
Task arithmetic	$0.685_{\pm 0.011}$	$0.503_{\pm 0.026}$	$0.552_{\pm 0.000}$	$0.130_{\pm 0.000}$	$0.602_{\pm 0.004}$	$0.117_{\pm 0.004}$
+XGD	$0.740_{\pm 0.004}$	$0.539_{\pm 0.015}$	$0.676_{\pm 0.012}$	$0.052_{\pm 0.042}$	$0.653_{\pm 0.007}$	$0.215_{\pm 0.032}$
Breadcrumbs	$0.705_{\pm 0.000}$	$0.521_{\pm 0.000}$	$0.470_{\pm 0.020}$	$0.132_{\pm 0.017}$	$0.603_{\pm 0.000}$	$0.119_{\pm 0.000}$
+XGD	$0.756_{\pm 0.006}$	$\mathbf{0.561}_{\pm 0.013}$	$0.667_{\pm 0.005}$	$0.114_{\pm 0.037}$	$0.658_{\pm 0.005}$	$0.225_{\pm 0.037}$
DARE-TIES	$0.708_{\pm 0.000}$	$0.486_{\pm 0.000}$	$0.554_{\pm 0.016}$	$0.256_{\pm 0.021}$	$0.520_{\pm 0.000}$	$0.202_{\pm 0.000}$
+XGD	$0.766_{\pm 0.001}$	$0.521_{\pm 0.003}$	$\mathbf{0.684}_{\pm 0.003}$	$0.181_{\pm 0.013}$	$0.662_{\pm 0.006}$	$\mathbf{0.287}_{\pm 0.022}$
TIES	$0.734_{\pm 0.007}$	$0.512_{\pm 0.014}$	$0.549_{\pm 0.000}$	$0.234_{\pm 0.000}$	$0.612_{\pm 0.013}$	$0.201_{\pm 0.013}$
+XGD	$\mathbf{0.774}_{\pm 0.004}$	$0.550_{\pm 0.011}$	$0.673_{\pm 0.006}$	$\mathbf{0.290}_{\pm 0.042}$	$\mathbf{0.679}_{\pm 0.010}$	$0.275_{\pm 0.006}$
θ_0+GD	$0.525_{\pm 0.036}$	$0.351_{\pm 0.032}$	$0.395_{\pm 0.048}$	$0.011_{\pm 0.064}$	$0.563_{\pm 0.016}$	$0.107_{\pm 0.013}$
θ_0+ungated	$0.548_{\pm 0.006}$	$0.402_{\pm 0.010}$	$0.456_{\pm 0.008}$	$0.066_{\pm 0.055}$	$0.578_{\pm 0.003}$	$0.151_{\pm 0.007}$
θ_0+XGD	$0.681_{\pm 0.050}$	$0.502_{\pm 0.020}$	$0.565_{\pm 0.032}$	$0.062_{\pm 0.051}$	$0.613_{\pm 0.056}$	$0.188_{\pm 0.083}$
Fisher (replay)	$0.658_{\pm 0.016}$	$0.401_{\pm 0.077}$	$0.509_{\pm 0.025}$	$0.021_{\pm 0.032}$	$0.607_{\pm 0.010}$	$0.113_{\pm 0.005}$
Localize-and-Stitch	$0.661_{\pm 0.014}$	$0.480_{\pm 0.009}$	$0.505_{\pm 0.107}$	$-0.063_{\pm 0.209}$	$0.584_{\pm 0.013}$	$0.134_{\pm 0.011}$

showed a different instability—a ±0.081 spread with the fine-grained Cars task collapsing on two draws of three—which we traced to the replay sampler: with more classes than the budget holds it had kept the lowest class ids deterministically, so Cars was refined and selected on 32 of its 196 classes, identically for every seed. Drawing the covered classes at random per seed (Appendix C) removes both the collapse and the spread. We report this because the failure is instructive: at these budgets a fine-grained task is representable only in part, and a selection buffer that inherits the same blind spot cannot detect damage to the classes it never sees.

Refining a merge. The same rule, unchanged, applies from a merge initialization, and this is where the refinement is most reliable: across all three suites at both budgets, every one of the twenty-four composition cells improves the merge it started from, on every draw. The best cell of each suite is a composition rather than the cold start—0.793 on CLIP-8 and 0.699 on GLUE-8 at $B = 32$, both from DARE-TIES, and 0.695 on CLIP-20 from TIES—and the gain is largest where the merge is weakest: 0.17 from DARE-TIES on CLIP-20 (0.520 → 0.692) and 0.19–0.21 from Breadcrumbs on GLUE-8 at $B = 16$ (0.470 → 0.667). Refinement also compresses the spread between merges, so which merge one starts from matters less once the labels are spent: CLIP-20's four merges span 0.520–0.619 before refinement and 0.664–0.695 after, and GLUE-8's at $B = 16$ span 0.470–0.554 before and 0.667–0.684 after. Composition is the more stable of the two settings at the smaller budget—on CLIP-8 at $B = 16$ the cold start varies by ±0.050 across draws while the four composition rows vary by ±0.001–0.006—and it removes the one cell in which a merge beats refinement. The selected rates are an order of magnitude smaller than from the pretrained model ($\eta \in [0.25, 4]$ on the CLIP suites against $\eta = 32$; several draws sit at the twice-extended lower edge of the grid), which is what a starting point that already carries multi-task signal should require. Worst-task behaviour splits by initialization on GLUE-8: TIES+XGD raises the weakest task from 0.234 to 0.363, while task arithmetic+XGD lowers it from 0.130 to 0.059. The ablation arms are reported only from the pretrained model, where they isolate the anchor and the gate; from a merge the comparison of interest is against the merge itself, which every composition row makes directly.

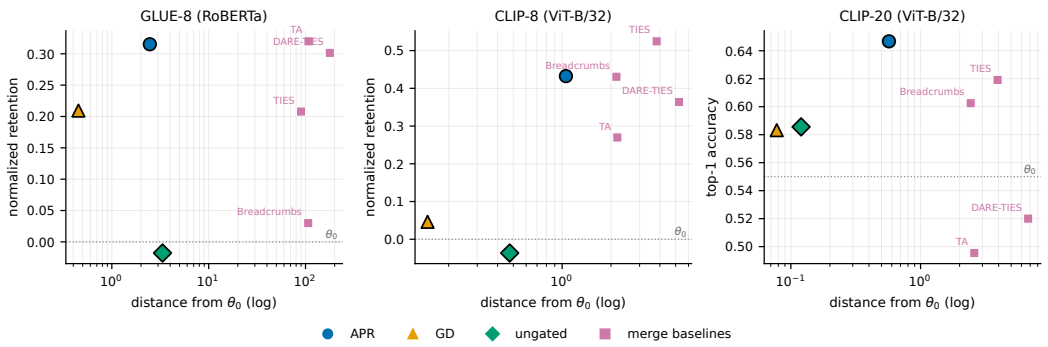

Figure 2: Task performance versus distance from the pretrained model θ_0. Circles, triangles, and diamonds show split-selected XGD, GD, and ungated refinement from θ_0; squares show the selected merge baselines. The horizontal dotted line marks pretrained performance, and distance is shown on a log scale.

Other uses of the same labels. Fisher merging with a replay-estimated Fisher, and Localize-and-Stitch with masks learned on the buffer, spend the same B labels per task on deciding how to weight the experts rather than on moving the encoder. Neither is competitive at this budget: across all six suite–budget cells they land below the best checkpoint-only merge every time—0.657 and 0.662 against TIES's 0.732 on CLIP-8 at $B = 32$, 0.541 and 0.487 against DARE-TIES's 0.563 on GLUE-8, 0.610 and 0.605 against TIES's 0.619 on CLIP-20—and below every composition row in every cell. Localize-and-Stitch is also the least stable estimator in the study: its GLUE-8 mean varies by ± 0.077 at $B = 32$ and ± 0.107 at $B = 16$, and its worst task there sits below the pretrained model. At this sample size the labels are better spent moving the encoder than estimating what to weight.

Twenty tasks. CLIP-20 is the high-interference regime the protocol was meant to probe. With twenty task vectors competing for one encoder, the best checkpoint-only merge recovers about a fifth of the floor-to-ceiling gap (0.619, against a 0.550 zero-shot floor and a 0.896 expert ceiling), where at eight tasks TIES recovers roughly half; Appendix E traces this to the geometry of the twenty-expert compromise. The selected task-arithmetic coefficient falls accordingly, to $\lambda \in \{0.05, 0.08\}$ against $\{0.2, 0.3\}$ at eight tasks. Refinement is correspondingly more useful there: the cold start clears the best merge by 0.044, and every composition improves its own initialization by 0.04–0.18. Pinning $S \in \{20, 100\}$ at the CLIP-20 composition cells confirms that the fixed horizon is not what limits them (Appendix **??**).

5.2 Retention of Pretrained Capabilities

Strong performance on the merged tasks is useful only if it does not erase capabilities inherited from the pretrained model. We test this first on the pretraining objective itself and then on tasks that are never merged or used for refinement.

Drift on the pretraining objective. Figure 2 shows the refinement reaching merge-level scores at a small fraction of the merge's displacement, but parameter distance is a geometric quantity, not a capability measure. We therefore measure drift on the pretraining objective itself: masked-language-model loss on WikiText-2 validation text. Each refined encoder is evaluated under RoBERTa's original pretrained MLM head; the head is never modified by any experiment, and the WikiText-2 text is never used for refinement or selection, so any increase in this loss over the pretrained model is attributable to encoder drift alone. The probed states are exactly the from-pretrained cells of Table 1 at $B = 32$—the saved winner of each of the three fresh draws, nothing re-derived—with the pretrained model and the merges at their selected hyperparameters as references (Table 3).

With the selected merges probed alongside (Table 3), the picture parallels the held-out-task study below. XGD is the only point that combines the best task score with sub-merge drift: at 0.612 it exceeds every merge (DARE-TIES 0.563, task arithmetic 0.552) while adding $+0.93$ to the pretraining loss against their $+1.59$ and $+2.51$—a better multitask model for 37% of task arithmetic's

Table 3: Pretraining-objective drift on GLUE-8 at the B=32 budget of Table 1. MLM loss is measured on WikiText-2 validation with RoBERTa's pretrained MLM head; Δ is relative to θ_0. Every probed state is the model Table 1 scores (the saved winner of each of the three fresh draws, and the merges at their selected hyperparameters), so the GLUE-8 column repeats that table's mean of per-task metrics; mean $\pm$ std over draws. * marks the score–drift Pareto frontier.

State	Dist. θ_0	GLUE-8 (task metric)	MLM loss (Δ)
Pretrained $\theta_0{}^*$	0.00	0.354	2.51 (0.00)
Task arithmetic (λ=0.3)	107.7	0.552 ± 0.000	5.02 (+2.51)
TIES*	90.0	0.549 ± 0.000	**2.95** (+0.44)
DARE-TIES	178.3	0.563 ± 0.000	4.10 (+1.59)
Breadcrumbs	101.0	0.473 ± 0.015	3.44 (+0.93)
XGD from $\theta_0{}^*$	3.54 ± 1.28	$\mathbf{0.612} \pm 0.015$	3.44 ± 0.31 (+0.93)
GD ablation	0.40 ± 0.01	0.521 ± 0.017	3.56 ± 0.07 (+1.05)
ungated ablation	1.95 ± 0.98	0.480 ± 0.060	7.89 ± 6.01 (+5.38)

Table 4: Held-out zero-shot retention after building models on CLIP-8 at B=32. Every state is the model Table 1 scores—the saved winner of each fresh draw, and the merges at their selected hyperparameters—evaluated with no further training on the seven CLIP-20 tasks on which θ_0 performs meaningfully above chance; parentheses report the change from θ_0; mean $\pm$ std over three draws. * marks the train–held-out Pareto frontier.

Model (built from the 8 train tasks)	Dist. θ_0	Train-8	Held-out (drop)
Base model $\theta_0{}^*$	0.00	0.478	0.780 (0.000)
GD from base*	0.04 ± 0.03	0.546 ± 0.032	$\mathbf{0.773} \pm 0.008$ (-0.006)
Ungated from base*	0.30 ± 0.06	0.579 ± 0.009	0.746 ± 0.009 (-0.034)
XGD from base*	1.05 ± 0.02	$\mathbf{0.751} \pm 0.004$	0.721 ± 0.009 (-0.059)
Task arithmetic	1.70 ± 0.42	0.685 ± 0.011	0.695 ± 0.058 (-0.085)
Breadcrumbs	2.16 ± 0.00	0.705 ± 0.000	0.716 ± 0.000 (-0.064)
TIES*	3.08 ± 0.04	0.732 ± 0.002	0.724 ± 0.030 (-0.056)
DARE-TIES	5.25 ± 0.00	0.708 ± 0.000	0.613 ± 0.000 (-0.166)

damage, at roughly 3% of its parameter displacement—and it dominates Breadcrumbs outright (the same drift at 0.14 more score). The exception is TIES, which drifts least of every merge (+0.44) at 0.063 less score, so the frontier consists of θ_0, TIES and XGD alone. The ablations locate the drift protection: ordinary descent barely moves (0.40 from θ_0) and still drifts more than XGD, while the ungated update, which moves less far than XGD on average, adds +12.3 to the loss on one draw of three—distance scaling without the gate can leave the encoder while barely displacing it. Refinement from the pretrained model is thus not the minimum-drift point, but it is the only one at the high-score end that costs the base model less than half of what a merge at that level does.

We also test whether the methods preserve zero-shot capabilities on tasks excluded from merging and refinement. Models are built on the standard CLIP-8 suite under the protocol of Table 1 at $B = 32$ and evaluated, with no further training or selection, on seven additional CLIP-20 tasks on which the pretrained model performs meaningfully above chance; every probed state is the saved winner of a fresh draw or a merge at its selected hyperparameters, so nothing is re-derived. The full twelve-task split and the five near-chance tasks are given in Appendix E.

XGD scores highest on the train tasks (0.751) and retains 0.721 of held-out accuracy (a drop of 0.059), which matches the best merge, TIES (0.724, inside a single draw's spread), at a third of its displacement and 0.019 more train accuracy; every other merge is dominated—Breadcrumbs (0.716), task arithmetic (0.695) and DARE-TIES (0.613) retain less at a lower train score. The two ablations retain more (0.773 and 0.746), but they do so by barely moving—ordinary descent stays within 0.04 of θ_0 and scores 0.21 less on the train tasks—so the frontier consists of θ_0, the two ablations, XGD and TIES, and retention is cheap for a model that has not learned the tasks. What the ablations do show is that XGD's retention is not a consequence of its labels alone: it gains 0.17–0.21 train accuracy over both while giving up 0.03–0.05 of held-out accuracy, whereas every merge but TIES gives up 0.06–0.17 for a lower train score.

Summary. Five claims survive the campaign at $n \leq 16$: (i) the refinement is broadly compositional, improving most initializations, while the small-buffer Breadcrumbs rows delimit any universal improvement claim; (ii) the anchored, distance-scaled, clipped step is the load-bearing ingredient, the gate contributing worst-task repair, small-budget stability, and held-out retention rather than mean accuracy; (iii) in the dedicated safety audit, anchored runs avoid the overfitting signature that appears in a quarter of unconstrained descent's small-buffer configurations, although selected composition rows can still decline; (iv) eight labels per task suffice to beat the tuned task-arithmetic merge starting from the pretrained model alone; and (v) on held-out tasks never merged or refined, the anchored update retains as much as the best checkpoint-only merge while scoring higher on the merged tasks, and more than every other merge; its unconstrained controls retain more only by staying near the pretrained model (Table 4).

6 Discussion

We introduced expert-anchored gradient descent (XGD), a refinement method designed to repair task interference in model merges. XGD uses replay gradients to identify locally beneficial movements toward each task expert, scales those movements by the expert distance, and clips them before they can overshoot. Under the equal-budget protocol XGD improves every checkpoint-only merge it is given, on every buffer draw, on both eight-task suites; when initialized directly from the pretrained model, it is the strongest of the tested replay refinements on all three benchmarks and on every buffer draw, and under the equal-budget protocol it outperforms every checkpoint-only merge on GLUE-8 at both budgets and on CLIP-8 at $B = 32$, trailing TIES on CLIP-8 at $B = 16$ because a learning rate selected on eight examples per task is conservative about the step the refit on sixteen can afford. The ablations identify the anchored, distance-scaled, clipped update as the main source of the gains, with the gate contributing primarily to worst-task performance and retention. On tasks excluded from merging and refinement, XGD retains as much zero-shot ability as the best checkpoint-only merge while scoring higher on the merged tasks, and more than every other merge; its unconstrained controls retain more only by staying near the pretrained model. XGD is therefore best viewed as a practical correction stage for model merging when a modest labeled buffer is available, with the option to operate directly from the pretrained model when no suitable merge exists. Establishing whether these conclusions extend beyond the encoder architectures and small-data regimes studied here remains future work.

AI use statement

In this work, we used generative AI tools to assist with writing and refactoring experiment code and with editing and polishing the text. We did not use generative AI tools for research ideation, experimental design, or data analysis. We have reviewed all AI-assisted work and take responsibility for the final content of this work, including any text, claims, or artifacts produced with the aid of generative AI.

References

Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*, 2016.

Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K. Dokania, Philip H. S. Torr, and Marc'Aurelio Ranzato. On tiny episodic memories in continual learning. *arXiv preprint arXiv:1902.10486*, 2019.

Mohammad Davari and Eugene Belilovsky. Model Breadcrumbs: Scaling multi-task model merging with sparse masks. *arXiv preprint arXiv:2312.06795*, 2023.

Pala Tej Deep, Rishabh Bhardwaj, and Soujanya Poria. DELLA-Merging: Reducing interference in model merging through magnitude-based sampling. *arXiv preprint arXiv:2406.11617*, 2024.

Shwai He, Runqi Wang, Jindong Wang, Jindong Gu, and Xing Xie. Localize-and-Stitch: Efficient model merging via sparse task arithmetic. *arXiv preprint arXiv:2408.13656*, 2024.

Yifei He, Siqi Zeng, Yuzheng Hu, Rui Yang, Tong Zhang, and Han Zhao. MergeBench: A benchmark for merging domain-specialized LLMs. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2025.

Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

Xisen Jin, Xiang Ren, Daniel Preotiuc-Pietro, and Pengxiang Cheng. Dataless knowledge fusion by merging weights of language models. In *International Conference on Learning Representations*, 2023.

Fanshuang Kong, Richong Zhang, and Ziqiao Wang. Activated parameter locating via causal intervention for model merging. *arXiv preprint arXiv:2408.09485*, 2024.

Janos Kramar, Tom Lieberum, Rohin Shah, and Neel Nanda. AtP*: An efficient and scalable method for localizing language model behavior to components. *arXiv preprint arXiv:2403.00745*, 2024.

Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *International Conference on Learning Representations*, 2022.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

Zhenyi Lu, Chengxu Yin, Wujie Wang, and Xuebo Liu. Twin-Merging: Dynamic integration of modular expertise in model merging. *arXiv preprint arXiv:2406.15479*, 2024.

Michael S. Matena and Colin A. Raffel. Merging models with Fisher-weighted averaging. In *Advances in Neural Information Processing Systems*, 2022.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021.

Anthony Robins. Catastrophic forgetting, rehearsal and pseudorehearsal. *Connection Science*, 7(2):123–146, 1995.

Wenju Sun, Qingyong Li, Wen Wang, Yangli-ao Geng, and Boyang Li. Task Arithmetic in Trust Region: A training-free model merging approach to navigate knowledge conflicts. *arXiv preprint arXiv:2501.15065*, 2025.

Aaquib Syed, Can Rager, and Arthur Conmy. Attribution patching outperforms automated circuit discovery. *arXiv preprint arXiv:2310.10348*, 2024.

Anke Tang, Li Shen, Yong Luo, Liang Ding, Han Hu, Bo Du, and Dacheng Tao. FusionBench: A comprehensive benchmark of deep model fusion. *arXiv preprint arXiv:2406.03280*, 2024.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*, 2018.

Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. SuperGLUE: A stickier benchmark for general-purpose language understanding systems. In *Advances in Neural Information Processing Systems*, 2019.

Ke Wang, Nikolaos Dimitriadis, Guillermo Ortiz-Jiménez, François Fleuret, and Pascal Frossard. Localizing task information for improved model merging and compression. In *International Conference on Machine Learning*, 2024.

Yongxian Wei, Anke Tang, Li Shen, Zixuan Hu, Chun Yuan, and Xiaochun Cao. Modeling multi-task model merging as adaptive projective gradient descent. In *International Conference on Machine Learning*, 2025.

Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, 2022.

Shitao Xiao, Zheng Liu, Peitian Zhang, and Xingrun Xing. LM-Cocktail: Resilient tuning of language models via model merging. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-Merging: Resolving interference when merging models. *arXiv preprint arXiv:2306.01708*, 2023.

Enneng Yang, Zhenyi Wang, Li Shen, Shiwei Liu, Guibing Guo, Xingwei Wang, and Dacheng Tao. AdaMerging: Adaptive model merging for multi-task learning. *arXiv preprint arXiv:2310.02575*, 2024.

Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are Super Mario: Absorbing abilities from homologous models as a free lunch. *arXiv preprint arXiv:2311.03099*, 2023.

Zihan Zeng, Jiahui Chen, Lei Li, and Min Zhang. Efficient model editing with task vector bases: A theoretical framework and scalable approach. *arXiv preprint arXiv:2502.01015*, 2025.

A Box-Distance Monotonicity and the Expert Box

This appendix proves the general geometric result behind Corollary 1. The statement is self-contained: $x_1, \dots, x_n$ are arbitrary points in $\mathbb{R}^D$ (in the application, the task experts $\theta_1, \dots, \theta_T$), and the superscript d indexes coordinates.

Theorem 1 (Box-distance monotonicity under coordinatewise movement toward a hull point)**.** *Let $x_1, \dots, x_n \in \mathbb{R}^D$. Define the axis-aligned box hull*

$$B := \prod_{d=1}^{D} [m_d, M_d], \qquad m_d := \min_{1 \le j \le n} x_j^d, \qquad M_d := \max_{1 \le j \le n} x_j^d.$$

Let $y, z \in \mathbb{R}^D$. Suppose that there exists some $i \in \{1, \dots, n\}$ such that, for every coordinate $d \in \{1, \dots, D\}$, the point z^d lies between x_i^d and y^d. Equivalently,

$$|z^d - x_i^d| \le |y^d - x_i^d|$$

and $z^d - x_i^d$ has the same sign as $y^d - x_i^d$, allowing equality.

Then, for every $1 \le p \le \infty$,

$$d_p(z, B) \le d_p(y, B),$$

where $d_p(u, B)$ denotes the distance from u to B with respect to the ℓ^p-norm.

Proof. For each coordinate d, set

$$I_d := [m_d, M_d].$$

Since $x_i \in B$, we have

$$x_i^d \in I_d$$

for every d.

Define the one-dimensional distance function

$$\delta_d(t) := \operatorname{dist}(t, I_d) = \inf_{s \in I_d} |t - s|.$$

We first prove the following elementary one-dimensional claim.

Claim. Let $I = [m, M] \subset \mathbb{R}$, let $c \in I$, and suppose that z lies between c and y. Then

$$\operatorname{dist}(z, I) \le \operatorname{dist}(y, I).$$

Indeed, if $y \in I$, then since $c \in I$ and I is an interval, every point between c and y also lies in I. Hence $z \in I$, so

$$\operatorname{dist}(z, I) = 0 = \operatorname{dist}(y, I).$$

If $y > M$, then

$$\operatorname{dist}(y, I) = y - M.$$

Since z lies between $c \in I$ and $y > M$, we have $z \le y$. If $z \le M$, then $\operatorname{dist}(z, I) = 0$. If $z > M$, then

$$\operatorname{dist}(z, I) = z - M \le y - M = \operatorname{dist}(y, I).$$

The case $y < m$ is symmetric. This proves the claim.

Now apply the claim coordinatewise with

$$I = I_d, \qquad c = x_i^d.$$

Because z^d lies between x_i^d and y^d, we obtain

$$\delta_d(z^d) \le \delta_d(y^d)$$

for every $d \in \{1, \dots, D\}$.

Next, distance to an axis-aligned box separates coordinatewise. Namely, for $1 \le p < \infty$,

$$d_p(u, B) = \left(\sum_{d=1}^{D} \delta_d(u^d)^p \right)^{1/p},$$

and for $p = \infty$,

$$d_\infty(u, B) = \max_{1 \le d \le D} \delta_d(u^d).$$

This follows because the closest point of B to u is obtained by clipping each coordinate u^d to the interval $I_d = [m_d, M_d]$.

Since

$$\delta_d(z^d) \le \delta_d(y^d)$$

for every d, monotonicity of the ℓ^p-norm on the nonnegative orthant gives

$$d_p(z, B) \le d_p(y, B).$$

This proves the theorem. □

The box hull B may look like a loose relaxation of the ordinary convex hull of the experts, but it is canonical in its own right: it is the smallest set containing the experts that is convex with respect to ℓ^1-geodesics.

Proposition 2. *The set B is the ℓ^1-geodesic convex hull of $\{x_1, \dots, x_n\}$.*

Proof. For $a, b \in \mathbb{R}^D$, define the ℓ^1-metric interval between a and b by

$$I_1(a, b) := \{u \in \mathbb{R}^D : \|a - u\|_1 + \|u - b\|_1 = \|a - b\|_1\}.$$

We claim that

$$I_1(a, b) = \prod_{d=1}^{D} [\min(a^d, b^d), \max(a^d, b^d)].$$

Indeed, by the triangle inequality in one dimension,

$$|a^d - u^d| + |u^d - b^d| \ge |a^d - b^d|$$

for each coordinate d. Summing over d, equality in the ℓ^1-triangle inequality holds if and only if equality holds in every coordinate. In one dimension, equality holds precisely when u^d lies between a^d and b^d. Hence the displayed formula for $I_1(a, b)$ follows.

Now B is clearly ℓ^1-geodesically convex: if $a, b \in B$, then every point coordinatewise between a and b also lies in B.

It remains to show minimality. Let $C \subseteq \mathbb{R}^D$ be any ℓ^1-geodesically convex set containing $x_1, \dots, x_n$. Since C is closed under ℓ^1-metric intervals, it is closed under taking coordinatewise boxes between pairs of its points. In particular, if $a, b \in C$, then both the coordinatewise minimum

$$a \wedge b := (\min(a^1, b^1), \dots, \min(a^D, b^D))$$

and the coordinatewise maximum

$$a \vee b := (\max(a^1, b^1), \dots, \max(a^D, b^D))$$

belong to C.

By induction, C therefore contains the coordinatewise minimum

$$m := (m_1, \dots, m_D)$$

and the coordinatewise maximum

$$M := (M_1, \dots, M_D)$$

of the points $x_1, \dots, x_n$. Since C is ℓ^1-geodesically convex, it contains the entire ℓ^1-metric interval between m and M. But

$$I_1(m, M) = \prod_{d=1}^{D} [m_d, M_d] = B.$$

Therefore every ℓ^1-geodesically convex set containing $x_1, \dots, x_n$ also contains B. Hence B is the smallest such set, i.e.

$$B = \mathrm{gconv}_{\ell^1}\{x_1, \dots, x_n\}.$$

□

We can now state and prove the guarantee for Algorithm 1 used in Section 3.2.

Corollary 1 (Box-distance monotonicity of the sequential refinement)**.** *Let*

$$B_{\exp} := \prod_{r=1}^{d} [\underline{\theta}_r, \overline{\theta}_r], \qquad \underline{\theta}_r := \min_{j \in \mathcal{T}} \theta_{j,r}, \qquad \overline{\theta}_r := \max_{j \in \mathcal{T}} \theta_{j,r},$$

be the axis-aligned box hull of the task experts. Then every within-sweep update of Algorithm 1 is non-expansive in distance to the expert box: for every $1 \le p \le \infty$*,*

$$d_p\Big(\theta^{(s,k)}, B_{\exp}\Big) \le d_p\Big(\theta^{(s,k-1)}, B_{\exp}\Big).$$

Consequently, after any number S *of sweeps,*

$$d_p\Big(\theta^{(S)}, B_{\exp}\Big) \le d_p\Big(\theta^{(0)}, B_{\exp}\Big) \qquad \textit{for every } 1 \le p \le \infty.$$

Proof. Fix a sweep s, a within-sweep index k, and let $i = \pi_s(k)$. We first verify the hypothesis of Theorem 1 with

$$y = \theta^{(s,k-1)}, \qquad z = \theta^{(s,k)}, \qquad x_i = \theta_i.$$

It is enough to check that, for every coordinate r, the new coordinate $\theta_r^{(s,k)}$ lies between the previous coordinate $\theta_r^{(s,k-1)}$ and the expert coordinate $\theta_{i,r}$.

By Equation (4), every coordinate update has the form

$$u_{i,r}^{(s,k)} = \alpha_{i,r}^{(s,k)} v_{i,r}^{(s,k)}, \qquad \alpha_{i,r}^{(s,k)} = \begin{cases} \min\big(\eta\, |g_{i,r}^{(s,k)}|,\, 1\big) & \text{if } g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < 0, \\ 0 & \text{otherwise,} \end{cases}$$

with $\alpha_{i,r}^{(s,k)} \in [0, 1]$ in both cases since $\eta \ge 0$. The same representation is assumed directly in the optional post-processing case. Hence

$$\theta_r^{(s,k)} = \theta_r^{(s,k-1)} + \alpha_{i,r}^{(s,k)} \big(\theta_{i,r} - \theta_r^{(s,k-1)}\big),$$

so $\theta_r^{(s,k)}$ lies between $\theta_r^{(s,k-1)}$ and $\theta_{i,r}$.

The box $B_{\exp}$ is exactly the axis-aligned hull of the expert points $\theta_1, \ldots, \theta_T$. Therefore Theorem 1 gives

$$d_p\Big(\theta^{(s,k)}, B_{\exp}\Big) \le d_p\Big(\theta^{(s,k-1)}, B_{\exp}\Big)$$

for every $1 \le p \le \infty$. Chaining this inequality over $k = 1, \ldots, T$ and over sweeps $s = 0, \ldots, S-1$ gives the final statement. □

B Method Details: Sequential Scheduling

This appendix details the scheduling of the per-task updates, summarized in Section 3.2.

A secondary design choice concerns how the per-task updates are scheduled within a refinement sweep. An aggregated expert-guided update would compute all u_i at $\theta^{(s)}$ and then apply $\eta \sum_i u_i$ at once; we instead apply each task's update immediately after computing it. This has two motivations. First, each update is applied at the same local model state on which its gradient, expert direction, and gate were computed. Second, the expert-distance cap in Equation (4) controls each single-task movement before the next task sees the modified model, rather than allowing several task vectors to accumulate into one larger coordinate step.

Because the clip is applied *after* the learning-rate scaling, the per-coordinate movement is bounded by $|v_{i,r}^{(s,k)}|$ for any η; a single update can never overshoot the expert in a coordinate no matter how large η is, and the learning rate controls only how many coordinates are driven to the cap. The sequential rule still does not guarantee Pareto improvement across tasks, because an update that helps task i can hurt task j; we therefore treat task ordering, cross-task interference, and worst-task retention as explicit diagnostics rather than as consequences of the gate alone. In practice, one can also normalize $u_i^{(s,k)}$ by tensor-wise RMS before applying the single-task update.

C Extended Experimental Protocol

This appendix expands the protocol of Section 4: benchmark positioning, secondary tracks, replay sampling, and the full baseline roster.

Benchmark settings. The multi-head GLUE-8 benchmark (CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, RTE, excluding WNLI as is common) (Wang et al., 2018) uses RoBERTa-base as the encoder backbone (Liu et al., 2019): the shared encoder is merged and evaluated with each task's own trained head.

Replay data. For refinement, we use $n \in \{8, 16\}$ replay examples per task in the main grids (the twenty-task budget study extends to $n = 32$; Appendix E), sampled from training data or from a held-out split separated from the final validation split. Sampling is deterministic given the buffer seed. For classification tasks the draw is class-balanced: each class contributes $\lfloor n/C \rfloor$ examples and the remainder is filled uniformly at random. When a task has more classes than the budget holds—Cars (196) and SUN397 (397) at every budget we use, and DTD, RESISC45 and GTSRB at $n = 32$—the buffer cannot represent every class, so n classes are drawn at random per buffer seed and contribute one example each. Coverage is then part of what varies across draws, and fine-grained tasks are the ones for which a small buffer is least representative. STS-B, a regression task, is sampled uniformly. The replay and selection buffers are drawn from a single index stream and split disjointly, so they can never overlap. Sensitivity to the replay-budget size and to the buffer draw is reported for each track, rather than only the best tuned replay size.

Protocol details. The selection objective is the mean over tasks of the held-out per-task metric (Section 4). On a small fold the metric is quantized (one example in $8 \cdot B/2$) and cells can tie exactly; a tie goes to the least-moving candidate—the smaller η for the refinement arms, the smaller scaling coefficient and then the smaller density for a merge. On the fresh draws such ties occurred in 9 of 54 refinement cells, none of them on GLUE-8, and in 2 of 80 merge selections. Appendix **??** records why the metric replaced the loss. Select-then-refit chooses η at construction size $B/2$ and

applies it at B; we accept this standard caveat rather than spend half the budget only on selection, and Appendix **??** measures what it costs at $B = 16$. A selected cell counts as interior when both neighbouring grid values are worse; when it lies on a boundary the grid is extended in that direction and selection repeated (at most twice). Refinement runs S sweeps at a constant learning rate with the task order re-randomized every sweep from a fixed seed, so the D draws vary only the buffer; the expert-distance cap is applied per update (Eq. (4)). The horizon is fixed at $S = 50$ sweeps for every refinement family (Appendix **??**), and each family receives an equal-size η grid: $\{1, \ldots, 32\}$ (XGD), $\{0.5, \ldots, 16\}$ (ungated), $\{10^{-5}, \ldots, 10^{-2}\}$ (GD) on the encoder suites, in factor-of-two steps, extended in the direction of a boundary selection. Buffer seeds 103–105 produce every reported draw; seeds 100–102 were used only to settle the protocol. AdaMerging uses the matched budget (the same B unlabeled inputs per task) rather than its transductive formulation on the evaluation split; DOGE/APGD builds its candidates without labels and uses the budget only to select its global scale. This accounting differs from common practice, where merge coefficients are tuned on the tasks' full validation splits outside any stated budget (Ilharco et al., 2022; Yadav et al., 2023) and that tuning dominates the cost of merging (He et al., 2025); here it happens inside the same B labels every method is charged for.

Normalized retention. Earlier versions of this study aggregated GLUE-8 by normalized retention,

$$\mathrm{NormRet}_t = \frac{\mathrm{Score}_t(\theta_{\mathrm{merge}}) - \mathrm{Score}_t(\theta_0)}{\mathrm{Score}_t(\theta_t) - \mathrm{Score}_t(\theta_0) + \epsilon_{\mathrm{metric}}}, \tag{5}$$

which places the pretrained model at 0 and the task expert at 1 ($\epsilon_{\mathrm{metric}}$ guards small denominators). We keep it for the per-task appendix tables but no longer use it as an aggregate, because its denominators are the trouble: on MRPC the expert improves on the pretrained model by only 0.168, so one point of accuracy there is worth six of retention and that task alone dominates worst-task columns, and on the twenty-task CLIP suite several experts barely improve on the zero-shot backbone (STL10 by 0.004), so a near-zero denominator would make per-task retention explode. The plain mean of per-task metrics that the body reports weights tasks by their own scales, which is the convention of the GLUE benchmark itself.

Per-task metrics. CoLA is scored by Matthews correlation, STS-B by Pearson/Spearman correlation, MNLI by matched/mismatched accuracy, MRPC and QQP by accuracy/F1, and the remaining GLUE tasks by accuracy; every CLIP task is scored by top-1 accuracy.

Baselines. Because the proposed method uses labeled replay buffers, baselines are grouped by the data that constructs them—hyperparameter selection is common to every method and uses the labeled selection buffer (Section 4): *checkpoint-only construction* (pretrained encoder, single-task experts, uniform averaging, task arithmetic, TIES, DARE, DELLA-style sampling, Model Breadcrumbs, magnitude-based Localize-and-Stitch, DOGE/APGD); *unlabeled construction* (RegMean, AdaMerging, APL); *labeled construction* (ordinary replay gradient descent from the same initialization, distance-scaled descent without the gate, Fisher merging with an empirical diagonal Fisher estimated from per-example labeled-loss gradients on the replay buffer, and Localize-and-Stitch with masks learned on the replay buffer); and *sanity controls* (inverted gate, density-matched random gate, wrong-expert directions, shuffled labels). All labeled-replay baselines use the same replay examples, the same number of gradient evaluations where possible, and the same hyperparameter search budget.

For DOGE/APGD we port the authors' released `DOGE_TA` implementation: two-dimensional encoder weights only; per-layer $\lambda_i^l = \eta / \|\tau_i^l\|$; task and shared SVD ranks $\min(d_l)/T$ and $\min(d_l)/6$; 400 Adam iterations at learning rate 10^{-4} with gradients projected orthogonally to the shared basis; and the released inclusive global top-30% task-vector mask applied to each adjusted vector. We sweep $\eta \in \{0.075, 0.10, 0.15, 0.20\}$ on GLUE-8, $\{0.03, 0.05, 0.07, 0.09, 0.11, 0.13, 0.15, 0.18\}$ on CLIP-8, and $\{0.0025, 0.005, 0.0075, 0.01, 0.015, 0.02, 0.025, 0.03, 0.05\}$ on CLIP-20. The last grid is an audited extension after the initial $\{0.03, \ldots, 0.18\}$ grid hit its lower boundary; the boundary run was stopped before evaluation, and only the winner of the expanded grid was evaluated.

D Per-Task Results at the 8+8 Budget

Tables 5 and 6 give the task-level scores underlying the GLUE-8 and CLIP-8 columns of the retired 8+8 split-selection grid. Each row is the cell selected using the disjoint eight-example selection

buffer; the evaluation scores below did not participate in selection. For GLUE-8, "score" denotes the benchmark metric used in the aggregate: Matthews correlation for CoLA, correlation for STS-B, and accuracy or accuracy/F1 for the remaining tasks. CLIP-8 reports top-1 accuracy.

E TWENTY-TASK CLIP: EXTENDED RESULTS

This appendix gives the full twenty-task campaign behind Section 5.1.

Held-out retention protocol. For Table 4, models are built on the standard eight-task CLIP suite and evaluated zero-shot on the twelve additional tasks in the twenty-task suite. The primary held-out score averages CIFAR-10, CIFAR-100, STL10, Flowers102, Oxford Pets, Food101, and FashionMNIST, the tasks on which the pretrained model performs meaningfully above chance; PCAM, FER2013, EMNIST, KMNIST, and RenderedSST2 are excluded from that average because their pretrained accuracies are at or near chance. Every data-dependent method receives the same eight inputs per training task, and each refinement uses the split-selected (η, S) from its CLIP-8 experiment. Only refinements from the pretrained model are reported, avoiding retention differences inherited from a merge initialization. For reference, transductive AdaMerging uses all $87{,}433$ evaluation inputs and reaches 0.759 training accuracy with a -0.031 held-out drop; it is excluded from the matched-budget table.

The vision track scales to the twenty-task suite of Wang et al. (2024) using the published fine-tuned ViT-B/32 encoders for all twenty tasks (every expert was validated to beat its zero-shot floor by a wide margin, including the three tasks whose class-name orderings we had to reconstruct: PCAM $0.556 \to 0.887$, FER2013 $0.376 \to 0.711$, RenderedSST2 $0.586 \to 0.713$).

Why RegMean is an awkward anchor. RegMean is the initialization at which the anchored update is least advantaged, and the reason is specific to how RegMean sits in parameter space. RegMean solves each linear layer by matching *outputs* on that layer's inputs, $W = (\sum_i G_i)^{-1} \sum_i G_i W_i$; in input directions with negligible activation variance the solution is essentially unconstrained, so W acquires large components that are functionally inert on in-distribution inputs. The resulting merge point is over 450 from the base model in parameter norm—larger than the encoder norm itself (336)—while scoring a healthy 0.723, which is only possible because most of that displacement lies in inference-invisible directions. Every ingredient of XGD reads the anchor $v_i = \theta_i - \theta$, so from this initialization v_i inherits that inflated, non-functional component: the step $-g \odot |v|$ scales noise on directions where $|v|$ is huge but the gradient is near zero, and the cap $|v|$ is widest exactly there—which is also why the usable rate collapses by an order of magnitude here. This yields a concrete prediction: anchoring on the functional part of v (equivalently, refining RegMean from a task-vector-defined anchor) should remove the handicap—an experiment we leave to future work. It also implies that parameter-space distance from base is a valid drift proxy for the task-vector-based methods but not for RegMean, whose functional drift must be measured in output space.

Limitations. Four gaps remain. The budget grid of Table **??** is single-seed, with five-seed replication only at its from-pretrained $n = 8$ cell, and the held-out retention study of Table 4 and the drift probe of Table 3 cover the three fresh draws of Tables 1 and 2 but at $B = 32$ only. On the twenty-task suite, the coefficient grid offered to split selection for task arithmetic ends at $\lambda = 0.2$, above that benchmark's optimum, so the selected merge is a boundary pick that lands below the zero-shot floor, and the from-task-arithmetic refinement rows of Table **??** inherit that starting point; a selection grid extended below $\lambda = 0.2$ would need those cells re-run. The alternative labeled baselines are re-run under split selection at the $16{+}16$ budget only (Table **??**); the $8{+}8$ grids carry XGD and its two ablations alone. The predicted functional-anchor fix for the RegMean initialization is untested. The held-out retention study is single-seed, at one budget, on one split, and that split is a comparatively low-interference regime—eight training tasks, where merging has less to repair and the strongest label-free merge is correspondingly hard to beat; whether the ordering against AdaMerging changes as interference grows is untested here. RegMean's retention is likewise budget-dependent: worst in the table at an eight-input Gram budget, it behaves quite differently at larger ones. A further limitation is one of scope rather than execution: the results reported here are confined to budgets of at most 32 labeled examples per task, so we make no claim about how the ordering between the anchored and unconstrained families behaves when labeled data is plentiful.

Table 5: GLUE-8 at the split-selected 8+8 budget: raw per-task evaluation scores for every nonempty GLUE-8 row of the 8+8 split-selection grid. The single-task-expert row evaluates each task's own expert and is included as a ceiling reference.

Initialization	Treatment	CoLA	SST-2	MRPC	STS-B	QQP	MNLI	QNLI	RTE
Pretrained	alone	0.000	0.509	0.748	−0.047	0.316	0.336	0.495	0.473
Single-task expert	alone	0.623	0.936	0.916	0.909	0.901	0.873	0.924	0.798
Pretrained	+GD	0.000	0.509	0.158	0.107	0.473	0.355	0.495	0.527
	+ungated	0.000	0.592	0.695	0.169	0.566	0.343	0.489	0.531
	+XGD	0.033	0.787	0.692	0.741	0.572	0.397	0.532	0.458
Task arithmetic	alone	0.130	0.845	0.719	0.529	0.371	0.698	0.648	0.477
	+GD	0.057	0.888	0.755	0.715	0.812	0.715	0.793	0.531
	+ungated	0.017	0.878	0.775	0.821	0.792	0.747	0.738	0.513
	+XGD	0.079	0.869	0.718	0.763	0.812	0.742	0.732	0.534
Breadcrumbs	alone	0.142	0.894	0.388	0.423	0.336	0.665	0.496	0.509
	+GD	−0.003	0.883	0.759	0.777	0.794	0.710	0.735	0.545
	+ungated	0.031	0.883	0.775	0.808	0.788	0.696	0.743	0.556
	+XGD	0.065	0.867	0.739	0.810	0.803	0.717	0.763	0.563
DARE-TIES	alone	0.268	0.886	0.690	0.681	0.366	0.571	0.591	0.451
	+GD	0.194	0.896	0.789	0.822	0.639	0.611	0.714	0.448
	+ungated	0.129	0.893	0.762	0.809	0.780	0.655	0.772	0.523
	+XGD	0.149	0.881	0.794	0.825	0.789	0.679	0.785	0.534
TIES	alone	0.195	0.862	0.712	0.286	0.330	0.461	0.570	0.458
	+GD	0.146	0.870	0.762	0.802	0.715	0.534	0.725	0.440
	+ungated	0.170	0.860	0.740	0.768	0.715	0.555	0.761	0.451
	+XGD	0.190	0.860	0.767	0.795	0.748	0.581	0.745	0.477
AdaMerging	alone	−0.018	0.532	0.243	0.770	0.307	0.774	0.882	0.686
	+GD	−0.049	0.577	0.253	0.682	0.312	0.758	0.874	0.661
	+ungated	−0.026	0.588	0.731	0.815	0.750	0.756	0.883	0.588
	+XGD	0.010	0.751	0.758	0.789	0.773	0.797	0.835	0.603

Table 6: CLIP-8 at the split-selected 8+8 budget: per-task top-1 accuracy for every nonempty CLIP-8 row of the 8+8 split-selection grid. The single-task-expert row evaluates each task's own expert and is included as a ceiling reference.

Initialization	Treatment	SUN397	Cars	RESISC	EuroSAT	SVHN	GTSRB	MNIST	DTD
Zero-shot	alone	0.632	0.597	0.582	0.450	0.315	0.325	0.482	0.439
Single-task expert	alone	0.749	0.783	0.949	0.991	0.972	0.989	0.996	0.794
Zero-shot	+GD	0.575	0.403	0.527	0.626	0.431	0.326	0.557	0.385
	+ungated	0.601	0.521	0.556	0.643	0.471	0.356	0.565	0.400
	+XGD	0.617	0.512	0.640	0.818	0.776	0.662	0.928	0.447
Task arithmetic	alone	0.570	0.553	0.628	0.734	0.778	0.685	0.961	0.472
	+GD	0.553	0.391	0.692	0.868	0.856	0.691	0.753	0.487
	+ungated	0.555	0.450	0.659	0.847	0.830	0.748	0.947	0.477
	+XGD	0.618	0.598	0.716	0.856	0.880	0.781	0.959	0.517
Breadcrumbs	alone	0.641	0.613	0.694	0.791	0.769	0.668	0.949	0.521
	+GD	0.637	0.605	0.734	0.862	0.836	0.749	0.946	0.522
	+ungated	0.617	0.521	0.735	0.860	0.826	0.747	0.940	0.502
	+XGD	0.619	0.511	0.652	0.850	0.863	0.685	0.930	0.477
DARE-TIES	alone	0.594	0.577	0.663	0.713	0.862	0.792	0.977	0.486
	+GD	0.063	0.011	0.275	0.496	0.239	0.092	0.291	0.232
	+ungated	0.392	0.018	0.301	0.517	0.738	0.472	0.841	0.303
	+XGD	0.646	0.538	0.773	0.867	0.902	0.860	0.971	0.522
TIES	alone	0.667	0.636	0.732	0.738	0.859	0.800	0.969	0.528
	+GD	0.666	0.630	0.779	0.876	0.898	0.848	0.960	0.539
	+ungated	0.655	0.305	0.762	0.828	0.877	0.820	0.950	0.514
	+XGD	0.676	0.581	0.787	0.866	0.891	0.844	0.962	0.549
AdaMerging	alone	0.627	0.652	0.661	0.849	0.834	0.760	0.977	0.469
	+GD	0.624	0.645	0.710	0.896	0.877	0.813	0.965	0.481
	+ungated	0.620	0.334	0.716	0.904	0.866	0.804	0.965	0.469
	+XGD	0.649	0.655	0.731	0.899	0.875	0.814	0.972	0.504